

ANALYSE PERSONNELLE DE LA DYNAMIQUE DU PROJET

BC2 — Coordonner une équipe projet

Mobilisation d'une équipe projet et conduite du changement

Épreuve individuelle — 20% de la note BC2

Nom et Prénom	Omar AKAKBA
Promotion	[Ta promotion — ex. 2024-2025]
Groupe projet	[Groupe X]
Spécialisation	DEV
Date de rendu	20/06/2026

Document individuel — 8 à 10 pages de texte — rendu après la présentation orale collective (épreuve 2.1)

TABLE DES MATIÈRES

1. Mon rôle dans le projet CYNA
 - 1.1 Poste, responsabilités et périmètre
 - 1.2 Organisation au quotidien avec le groupe
2. Ma contribution technique et mes apprentissages
 - 2.1 Livrables et productions personnels
 - 2.2 Valeur ajoutée spécifique
 - 2.3 Ce que j'ai appris des autres membres
3. Dynamique collective : ce qui a fonctionné

3.1 Facteurs d'efficacité collective

3.2 Prise en compte des différences entre membres

3.3 Gestion des désaccords

4. Ce qui a été difficile

4.1 Principale difficulté technique

4.2 Principale difficulté organisationnelle ou relationnelle

4.3 Les moments où j'ai été un frein

4.4 Ce que ces difficultés ont changé

5. Bilan critique et enseignements

5.1 Vision du projet abouti

5.2 Ce que j'aurais fait différemment

5.3 Enseignements durables

5.4 Projection professionnelle

1. MON RÔLE DANS LE PROJET CYNA

1.1 Poste, responsabilités et périmètre

J'ai occupé le rôle de Lead Développeur Back-End sur le projet CYNA, une plateforme SaaS de cybersécurité permettant à des entreprises d'acheter des solutions EDR, SOC Managé et VPN en abonnement.

Mon périmètre couvrait l'intégralité du backend : authentification sécurisée (connexion, inscription, reset mot de passe, remember me, CSRF, rate limiting), catalogue produits, panier, tunnel de commande, intégration Stripe (Checkout + Webhooks), espace client (commandes, adresses, profil, suppression RGPD) et panel d'administration complet. J'étais directement responsable de la base de données MySQL — schéma de 10 tables, requêtes PDO, contraintes de clés étrangères.

En fin de projet, mon périmètre s'est élargi aux livrables d'infrastructure exigés par le BC3 : pipeline CI/CD GitHub Actions, 29 tests PHPUnit automatisés, infrastructure Docker, et monitoring Prometheus + Grafana avec alertes email.

1.2 Organisation au quotidien avec le groupe

L'organisation reposait sur GitHub comme outil central : chaque fonctionnalité faisait l'objet d'une branche dédiée, les commits étaient préfixés (`feat:` , `fix:` , `docs:`), et les échanges techniques passaient par les issues du dépôt. Les discussions quotidiennes se faisaient via [Discord / WhatsApp], avec des points de synchronisation [fréquence] pour vérifier l'avancement de chacun.

La répartition des tâches s'est organisée par domaine de compétence dès le début : je prenais en charge le backend et la base de données, pendant que [Prénom(s)] se concentrait/aient sur [leur(s) domaine(s)]. Un tableau [Trello / GitHub Projects] servait de référence partagée pour visualiser les tâches en cours et à faire.

2. MA CONTRIBUTION TECHNIQUE ET MES APPRENTISSAGES

2.1 Livrables et productions personnels

Ma première contribution a été la conception du schéma de base de données MySQL (10 tables, fichier `sql/schema.sql`), avec des contraintes RGPD intégrées dès le départ : la table `order` accepte un `user_id` nullable pour que la commande soit anonymisée — et non supprimée — lors de la suppression d'un compte, conformément à l'article 17 du RGPD.

J'ai ensuite livré l'intégration Stripe complète : Checkout Session pour le paiement, Webhook signé pour la mise à jour automatique du statut de commande, et gestion des cas d'échec. En parallèle, j'ai implémenté le système d'authentification avec hachage bcrypt, rate limiting par IP en base de données contre le brute force, et protection CSRF sur tous les formulaires.

En fin de projet, j'ai livré le pipeline CI/CD GitHub Actions (tests automatiques à chaque push), 29 tests PHPUnit répartis en trois classes (SecurityTest, AuthTest, DatabaseTest), et un endpoint `metrics.php` exposant 7 métriques applicatives à Prometheus, visualisées sur un dashboard Grafana avec 3 alertes email configurées.

2.2 Valeur ajoutée spécifique

Ma valeur ajoutée principale a été la prise en charge du CI/CD et du monitoring, deux domaines que personne dans le groupe ne maîtrisait. Quand le référentiel BC3 a exigé ces livrables, je les ai mis en place de zéro, en résolvant seul les neuf échecs successifs du pipeline CI avant d'obtenir un run vert — compétence entièrement développée pendant le projet.

J'ai également apporté une rigueur systématique sur la sécurité : zéro concaténation SQL (requêtes préparées PDO uniquement), échappement de toutes les sorties HTML via une fonction `escape()` centralisée, et tokens CSRF régénérés à chaque formulaire. Ces choix ont protégé l'application contre les vulnérabilités OWASP Top 10 les plus courantes et constituent une démonstration concrète de développement sécurisé.

2.3 Ce que j'ai appris des autres membres

[Prénom] m'a aidé à prendre du recul sur l'expérience utilisateur : plusieurs fonctionnalités que je considérais terminées techniquement ne guidaient pas suffisamment l'utilisateur. Cette confrontation entre logique back-end (le traitement est correct) et logique front-end (l'utilisateur comprend ce qui se passe) m'a appris à anticiper les messages d'erreur et les états intermédiaires dès la conception, pas en correction.

De [Prénom], j'ai appris la rigueur de la communication de l'avancement : là où je considérais une tâche terminée dès qu'elle fonctionnait en local, il/elle systématisait la mise à jour du tableau de suivi et la rédaction de commits explicites. En adoptant cette habitude, j'ai réduit les malentendus lors des points de synchronisation et rendu le dépôt GitHub plus lisible pour tout le groupe.

3. DYNAMIQUE COLLECTIVE : CE QUI A FONCTIONNÉ

3.1 Facteurs d'efficacité collective

Le facteur le plus déterminant a été la séparation claire des domaines de responsabilité : chaque membre était souverain sur sa partie, sans validation systématique des autres pour chaque commit. Je n'avais pas besoin d'attendre l'accord du groupe pour corriger un bug d'authentification — je livrais, et le reste du groupe était informé via le commit. Cela a supprimé la plupart des blocages liés à la dépendance décisionnelle.

Ce mécanisme a fonctionné parce qu'il reposait sur une confiance mutuelle dans les compétences de chacun. Quand j'ai proposé de remplacer Docker Desktop par OrbStack après le blocage macOS Sequoia, le groupe a accepté sans débat parce que c'était ma partie. Inversement, quand [Prénom] a revu l'organisation d'une page, je n'ai pas interféré. Cette délégation par domaine de compétence est ce qui a le plus contribué à la vitesse d'exécution du groupe.

Nous avons aussi utilisé GitHub non seulement comme outil de versioning mais comme mémoire technique : les commits préfixés permettaient à n'importe quel membre de comprendre ce qui avait changé sans lire le diff complet. Lors du débogage du pipeline CI, cette traçabilité a permis d'identifier précisément à quel commit un comportement avait changé.

3.2 Prise en compte des différences entre membres

Le groupe présentait des niveaux différents selon les domaines. Pour éviter le déséquilibre, nous avons adopté une règle implicite : quand quelqu'un prenait une tâche hors de sa zone de confort, le membre plus expérimenté dans ce domaine restait disponible pour débloquer sans prendre le relais. Par exemple, quand [Prénom] rencontrait une difficulté côté serveur, je donnais une explication et un exemple plutôt que de corriger moi-même — ce qui lui permettait de monter en compétence sans se sentir mis à l'écart.

En pratique, la charge n'était pas répartie de manière arithmétiquement égale mais de manière adaptée à ce que chacun pouvait produire de qualité. J'ai parfois pris plus de tâches techniques parce que c'était le chemin le plus rapide, sans que cela crée de ressentiment — la contribution de chaque membre sur d'autres plans (rédaction, présentation, coordination) était reconnue.

3.3 Gestion des désaccords

Un désaccord concret a porté sur le choix du système de paiement : une partie du groupe préférait PayPal pour sa simplicité d'intégration, tandis que je défendais Stripe pour sa gestion native des webhooks et sa documentation de qualité. Le désaccord portait sur un arbitrage réel entre facilité d'usage et robustesse technique.

Nous avons tranché en demandant à chacun de présenter un argument factuel : j'ai montré comment les webhooks Stripe gèrent les états de paiement de manière fiable, y compris les cas d'échec et de timeout. [Prénom] a reconnu que la robustesse était plus importante qu'une interface familière pour une plateforme SaaS. Le groupe a retenu Stripe. Ce processus m'a appris que les désaccords techniques se résolvent mieux par la démonstration que par la conviction.

4. CE QUI A ÉTÉ DIFFICILE

4.1 Principale difficulté technique

La mise en place du pipeline CI/CD GitHub Actions a été la principale difficulté technique : le pipeline a échoué neuf fois consécutives avant de passer au vert. Chaque échec révélait un problème différent à diagnostiquer uniquement via les logs d'une machine distante Ubuntu. Le premier problème était l'absence d'extension Xdebug sur le runner, qui bloquait la génération de couverture de code — résolu en désactivant complètement la couverture (`coverage: none`, `--no-coverage`).

Le deuxième problème était un mismatch de nom de base de données : le service MySQL créait `cyna_db` mais les variables d'environnement pointaient vers un nom différent. Le troisième était une erreur de clé dupliquée (`Duplicate entry 'edr'`) car `schema.sql` et `seed.sql` inséraient tous deux les mêmes catégories — résolu en remplaçant tous les `INSERT INTO` par `INSERT IGNORE INTO` dans les deux fichiers.

L'impact sur le groupe a été limité car je gérais cette tâche seul, mais plusieurs heures de débogage sur un environnement non contrôlable directement ont été un facteur de stress réel. J'ai retenu qu'un pipeline CI doit être construit de manière incrémentale — valider d'abord la connexion à la base, puis les migrations seules, puis les tests — plutôt qu'en empilant toutes les étapes dès le premier run.

4.2 Principale difficulté organisationnelle ou relationnelle

La principale difficulté organisationnelle a été la convergence simultanée de plusieurs livrables en fin de projet : stabilisation du pipeline CI, rédaction du DAT, génération du PDF, et production des captures d'écran. Certains livrables documentaires dépendaient d'éléments techniques non encore finalisés (pipeline vert, dashboard Grafana fonctionnel), ce qui créait des blocages en cascade.

Cette situation a mis en évidence un manque de planification des dépendances entre tâches techniques et tâches documentaires. Nous aurions dû définir une phase de gel des fonctionnalités quelques jours avant la remise, dédiée uniquement à la documentation. À la place, modifications techniques et rédaction ont coexisté jusqu'au dernier moment, générant une incertitude sur la version finale à livrer.

4.3 Les moments où j'ai été un frein

J'ai représenté un frein d'abord par une mauvaise estimation du temps : la mise en place du monitoring Prometheus + Grafana, annoncée comme un livrable d'une demi-journée, a pris deux jours complets en raison d'un bug dans la syntaxe PHP d'un en-tête HTTP qui a causé

une erreur Apache 500 pendant plusieurs heures. Ce décalage a mis [Prénom] en attente pour des captures d'écran du dashboard nécessaires au DAT.

J'ai également été un frein par mon mode de communication : face aux neuf échecs du pipeline CI, je cherchais la solution seul sans informer le groupe de l'état du blocage. Les autres membres n'avaient pas de visibilité sur ce qui se passait et ne pouvaient pas réorganiser leur travail en conséquence. Ce comportement de silence jusqu'à la solution est un défaut que j'ai identifié et que je dois corriger.

Enfin, j'ai pris certaines décisions techniques structurantes sans concertation préalable — le remplacement de Docker Desktop par OrbStack, le choix de PHPUnit 11 plutôt que 13. Ces choix étaient justifiés techniquement, mais les annoncer comme des faits accomplis a pu donner l'impression que je considérais ces décisions comme exclusivement de mon ressort, ce qui n'est pas compatible avec un fonctionnement d'équipe.

4.4 Ce que ces difficultés ont changé

Ces difficultés ont produit deux changements concrets. Le premier : toute tâche qui dépasse une journée sans résolution doit être signalée au groupe à mi-journée, que la solution soit trouvée ou non. Ce signal d'avancement — même négatif — permet aux autres d'anticiper et de réorganiser sans attendre que le blocage devienne une urgence collective.

Le second : je planifie désormais les livrables documentaires comme des dépendances de sprint au même titre que les tâches techniques. Dans un prochain projet, je réserverai une fenêtre dédiée aux captures d'écran et preuves visuelles avant la date de rendu, plutôt que de les collecter dans les dernières heures sous pression.

5. BILAN CRITIQUE ET ENSEIGNEMENTS

5.1 Vision du projet abouti

Ce qui me satisfait est la cohérence technique d'ensemble : la plateforme est fonctionnelle de bout en bout, du catalogue au paiement Stripe. Le schéma de base de données a tenu sans modification majeure du début à la fin, et le pipeline CI/CD avec 29 tests automatiques représente un niveau de maturité technique que je n'aurais pas atteint sans les exigences du référentiel BC3.

Ce qui me déçoit est l'absence de tests couvrant les parcours utilisateurs complets. Les tests PHPUnit valident des fonctions isolées mais ne testent pas le flux entier d'un achat (panier → commande → paiement Stripe → confirmation). En production, ce serait une lacune importante pour détecter les régressions sur le chemin critique de l'application.

Je suis également déçu par la couverture de la conformité RGPD : la suppression de compte est fonctionnelle, mais l'export des données personnelles (article 20 — droit à la portabilité) n'a pas été testé en conditions réelles. Pour une plateforme qui se positionne sur la cybersécurité et revendique la conformité RGPD comme argument commercial, c'est une contradiction directe avec le positionnement produit.

5.2 Ce que j'aurais fait différemment

Sur le plan individuel, j'aurais mis en place le pipeline CI/CD dès le début du projet. Configurer GitHub Actions sur un projet vide prend moins d'une heure ; le configurer sur un projet mature avec un schéma SQL complexe prend plusieurs jours de débogage. Si le pipeline avait existé dès le premier sprint, chaque ajout aurait été validé automatiquement et les problèmes de compatibilité auraient été détectés immédiatement.

Sur le plan collectif, j'aurais proposé un rituel de synchronisation quotidien court — quinze minutes — pour partager les blocages en cours. Ce format stand-up aurait permis d'identifier les dépendances bloquantes plus tôt et d'éviter les situations où un membre attendait une livraison bloquée depuis deux jours sans que personne ne le sache. Nous avons les outils mais pas le rituel, et c'est le rituel qui crée la discipline de communication.

5.3 Enseignements durables

Sur le plan technique, j'ai appris à déployer un pipeline CI/CD complet, à diagnostiquer des erreurs dans un environnement distant, et à configurer un système de monitoring applicatif avec Prometheus et Grafana. Ces trois compétences, que je ne maîtrisais pas en début de projet, sont directement transposables dans tout environnement de production professionnel.

Sur le plan humain, l'enseignement le plus durable est la distinction entre savoir faire et faire comprendre ce qu'on fait. J'ai produit des livrables techniquement corrects mais j'ai parfois manqué de transparence sur le chemin pour y arriver. En contexte professionnel, un développeur qui produit sans communiquer est difficile à intégrer dans une équipe. La communication sur le travail en cours est une compétence à part entière.

5.4 Projection professionnelle

Les compétences acquises sur CYNA que je vais mobiliser directement en entreprise sont, par ordre de priorité : CI/CD (attendu dès le premier jour dans la plupart des équipes Back-End), tests automatisés, et monitoring applicatif. Ces trois savoir-faire, je peux les démontrer avec le dépôt GitHub du projet comme preuve tangible.

Sur le plan comportemental, j'adopterai deux réflexes dans mon prochain projet : documenter les décisions techniques au moment où elles sont prises, et signaler les blocages dès qu'ils dépassent deux heures sans résolution. Ces deux comportements différencient un technicien d'un coordinateur — le technicien résout le problème, le coordinateur s'assure que tout le monde sait où en est la résolution. C'est cette posture que le projet CYNA m'a appris à construire.